

UNIVERSITAT POLITÈCNICA DE CATALUNYA

Escola Tècnica Superior d'Enginyeria de Telecomunicacions de Barcelona (ETSETB)

Practice 2

Grau en Enginyeria Física

Authors: Arnau Torret & Arnau Montull

Subject: SIMCOM

Date: October 30, 2026

1 Exercise 1

1.1 Question

Simulation of a simple Lennard-Jones fluid, such as liquid argon, with a hand-made FORTRAN MD code which incorporates the Leap-frog Verlet algorithm.

STEP 1: Understanding the code.

- Open the FORTRAN file `leapfroglj.f` and have a look.
- Note that you'll need 2 auxiliary files: (1) `leap-lj.data` and (2) `leap-conf.data`, containing (1) data input and (2) initial coordinates and velocities, respectively.
- The main program is ready to simulate a LJ liquid of 500 atoms of mass 39.948 uma, $\sigma = 3.405$ Å and $\varepsilon/k_B = 119.8$ K (Argon), with a time step of $\Delta t = 0.01$ ps. Remember that the LJ potential is of the form:

$$V_{\mathrm{LJ}}(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]. \tag{1}$$

- Compile the code and make a run of 10000 time steps. If you compute the temperature T and total energy E (kinetic + potential) you will get the following figures.

1.2 Code

For this step we simply run the suplied leap-frog program.We just added the accumulation and printing of the averages. The run uses the parameters of the statement (10000 steps, $N = 500$). The full program, with every later addition, is listed in the Annex.

```
53    sumt = 0.d0
54    sumt2 = 0.d0
55    sume = 0.d0
56    sume2 = 0.d0
57
58    ! 5. Start the loop to generate new configurations
59    do i = 1, nconf
60       call forces(natoms, r, boxlength, accel, rc, epot)
61       call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, &
62                   boxlength)
63       etot = ecin + epot
64       write(3,*) i*deltat, etot
65       write(4,*) i*deltat, temp
66       sumt = sumt + temp
67       sumt2 = sumt2 + temp*temp
68       sume = sume + etot
69       sume2 = sume2 + etot*etot
70    end do
71    close(3)
72    close(4)
73
74    ! Mean and standard deviation of T and E_tot over the run
75    tmean = sumt/nconf
76    emean = sume/nconf
77    tsd = dsqrt(sumt2/nconf - tmean**2)
78    esd = dsqrt(sume2/nconf - emean**2)
79    print *, 'steps  = ', nconf
80    print *, '<T>    = ', tmean, '  sigma_T = ', tsd
81    print *, '<Etot> = ', emean, '  sigma_E = ', esd
```

Listing 1: Main loop with the running sums of T and E_{tot} (added) and the printed means and standard deviations.

1.3 Results

With $L^* = 8.427$ ($\rho^* = 0.835$) and $\Delta t^* = 4.637 \times 10^{-3}$ ($\tau = 2.156$ ps), the 10000 steps cover $t^* = 46.37$ (100 ps). Figure 1 shows T and $E_{\text{tot}} = E_{\text{kin}} + E_{\text{pot}}$.

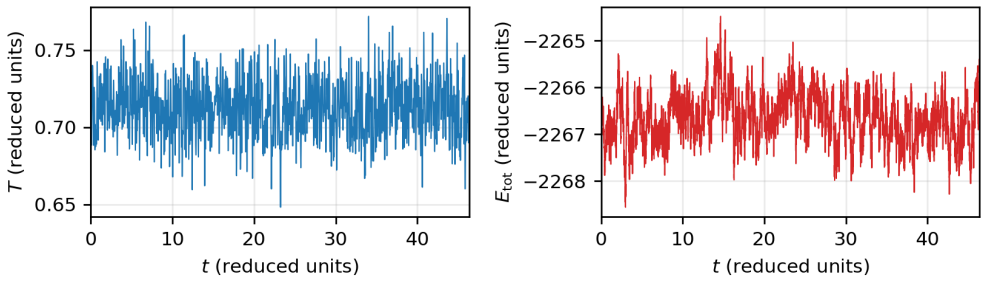


Figure 1: Temperature $T = 2E_{\text{kin}}/(3N - 3)$ (left) and total truncated energy E_{tot} (right), in reduced units.

1.4 Discussion & Notes

- **Energy:** leap-frog Verlet is symplectic, so E_{tot} should only oscillate. We get $\langle E_{\text{tot}} \rangle = -2266.65$, and $\sigma_E = 0.537$ (0.024%), a very small fluctuation.
- **Temperature:** $\langle T^* \rangle = 0.713$ (85.4 K), $\sigma_T = 0.0175$ (2.5%). That is a much larger relative fluctuation than for E_{tot}, as E_{kin} and E_{pot} are exchanged. Both curves match the reference figures for a dense liquid.

2 Exercise 2

2.1 Question

STEP 2: Analysing the structure and calculating thermodynamic properties.

- Modify the code to calculate the radial distribution function (RDF) $g(r)$ from the MD simulation. Compute the RDF for a run of 10000 steps. Plot $g(r)$ vs. r using reduced units.
- Modify the code to calculate the pressure P. Compute the mean values of the total and different contributions to E and P. Include the tail corrections. Analyze the convergence of the results as a function of the simulation length. Use reduced units.

2.2 Code

Only the lines added for the radial distribution function, the virial pressure and the long-range tails are shown. The full program is listed in the Annex.

```
9     double precision mass, nid
10
11    ! 1. Defining dimensions
12    dimension r(3,1000), vinf(3,1000), accel(3,1000)
13    parameter (nhis = 1000)
14    dimension g(nhis)
```

Listing 2: Histogram storage. `nid` is real because a name starting with `n` would be an integer under implicit typing.

```
49    pi = 4.d0*datan(1.d0)
50    delg = boxlength/(2.d0*dfloat(nhis))
51    rho = dfloat(natoms)/boxlength**3
52    etail = (8.d0*pi/3.d0)*rho*dfloat(natoms)* &
53            (1.d0/(3.d0*rc**9) - 1.d0/rc**3)
54    ptail = (16.d0*pi/3.d0)*rho**2* &
55            (2.d0/(3.d0*rc**9) - 1.d0/rc**3)
56    do j = 1, nhis
57       g(j) = 0.d0
58    end do
59    ekin_sum = 0.d0
60    epot_sum = 0.d0
61    pkin_sum = 0.d0
62    pvir_sum = 0.d0
```

Listing 3: Tail corrections and zeroing of the RDF and pressure accumulators before the loop.

```
74        ! Conserved NVE energy (no tail). Thermodynamic totals include it.
75        etot = ecin + epot
76        etot_corr = etot + etail
77        pkin = 2.d0*ecin/(3.d0*boxlength**3)
78        pvir = vir/(3.d0*boxlength**3)
79        ptot = pkin + pvir + ptail
80        ekin_sum = ekin_sum + ecin
81        epot_sum = epot_sum + epot
82        pkin_sum = pkin_sum + pkin
83        pvir_sum = pvir_sum + pvir
84        write(3,*) i*deltat, etot
85        write(4,*) i*deltat, temp
86        write(8,*) i*deltat, etot_corr, pkin, pvir, ptail, ptot
```

Listing 4: Energy and pressure contributions accumulated on every step of the MD loop.

```
92    ekin_avg = ekin_sum/dfloat(nconf)
93    epot_avg = epot_sum/dfloat(nconf)
94    etot_avg = ekin_avg + epot_avg + etail
95    pkin_avg = pkin_sum/dfloat(nconf)
96    pvir_avg = pvir_sum/dfloat(nconf)
97    ptot_avg = pkin_avg + pvir_avg + ptail
```

Listing 5: Run averages of the energy and pressure contributions.

```
120   ! 6. Radial distribution function in reduced units.
121   open(15, file='g-leap.dat', status='unknown')
122   do j = 1, nhis
123      rr = delg*(dfloat(j) - 0.5d0)
124      vb = (dfloat(j)**3 - dfloat(j-1)**3)*delg**3
125      nid = (4.d0/3.d0)*pi*vb*rho
126      g(j) = g(j)/(dfloat(nconf)*dfloat(natoms)*nid)
127      write(15,*) rr, g(j)
128   end do
129   close(15)
```

Listing 6: Normalisation of $g(r)$ by the ideal-gas content of each shell.

```
183   epot = 0.d0
184   vir = 0.d0
185
186   ! atom-atom interactions
187   do is = 1, natoms-1
188      do js = is+1, natoms
189         call lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
190         epot = epot + pot
191         vir = vir + virij
192      end do
193   end do
```

Listing 7: Pair virial summed inside `forces`.

```
230      virij = forcedist*rr2
231   end if
232
233   ! Histogram of pair distances up to L/2 (minimum image).
234   if (rr .lt. boxlength/2.d0) then
235      ig = int(rr/delg) + 1
236      if (ig .ge. 1 .and. ig .le. nhis) g(ig) = g(ig) + 2.d0
237   end if
```

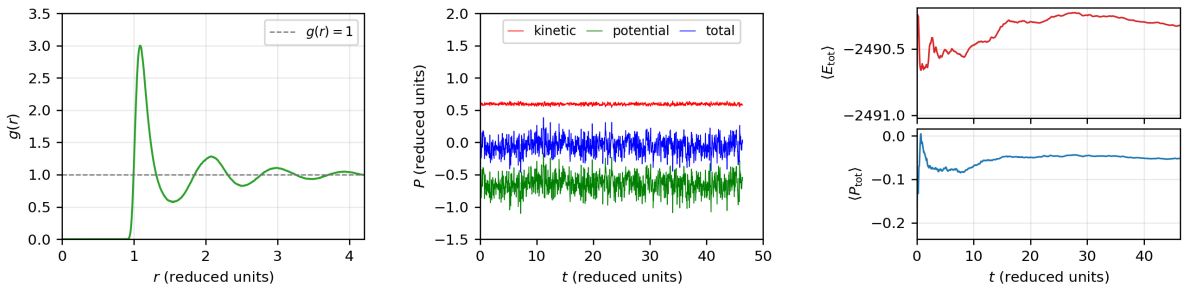
Listing 8: End of subroutine `lj`: virial of one pair inside the cutoff, and the $g(r)$ shell counted up to $L/2$.

2.3 Results

The same 10000-step trajectory as in Exercise 1 ($N = 500$, $\rho^* = 0.835$, $\langle T^* \rangle = 0.713$, $r_c = 2.5$) was used. $g(r)$ is accumulated in 1000 shells up to $L^*/2 = 4.21$ and normalised by the ideal-gas content of each shell. The pressure is $P = P_{\text{kin}} + P_{\text{vir}} + P_{\text{tail}}$ with $P_{\text{kin}} = 2E_{\text{kin}}/(3V)$ and $P_{\text{vir}} = \frac{1}{3V}\sum_{i<j} \vec{r}_{ij} \cdot \vec{f}_{ij}$. The tails assume $g(r) = 1$ beyond r_c:

$$E_{\text{tail}} = \frac{8\pi}{3}\rho N\left(\frac{1}{3r_c^9} - \frac{1}{r_c^3}\right) = -223.67, \qquad P_{\text{tail}} = \frac{16\pi}{3}\rho^2\left(\frac{2}{3r_c^9} - \frac{1}{r_c^3}\right) = -0.7465. \tag{2}$$

Figure 2 shows $g(r)$, the pressure contributions and the running means of the totals. Table 1 gives the mean of every contribution after increasing simulation lengths. The last row is the full run.



(a) RDF (b) Pressure (c) Convergence

Figure 2: Structure and thermodynamics of the Lennard-Jones liquid in reduced units. (a) Radial distribution function $g(r)$. (b) Kinetic, potential ($P_{\mathrm{vir}} + P_{\mathrm{tail}}$) and total pressure versus time. (c) Running means of E_{tot} and P_{tot} (tails included) from $t = 0$ to t.

Table 1: Mean energy and pressure contributions (reduced units) after n steps. $E_{\mathrm{tail}} = -223.67$ and $P_{\mathrm{tail}} = -0.7465$ are constant and are included in the totals.

n	t^*	$\langle E_{\mathrm{kin}} \rangle$	$\langle E_{\mathrm{pot}} \rangle$	$\langle E_{\mathrm{tot}} \rangle$	$\langle P_{\mathrm{kin}} \rangle$	$\langle P_{\mathrm{vir}} \rangle$	$\langle P_{\mathrm{tot}} \rangle$
1000	4.64	535.49	−2802.35	−2490.53	0.5965	0.0721	−0.0779
2500	11.59	534.82	−2801.60	−2490.45	0.5958	0.0922	−0.0585
5000	23.19	534.06	−2800.66	−2490.27	0.5949	0.1042	−0.0473
7500	34.78	533.35	−2799.94	−2490.26	0.5941	0.1061	−0.0462
10000	46.37	533.66	−2800.31	−2490.32	0.5945	0.1001	−0.0519

2.4 Discussion & Notes

- **RDF:** $g(r) = 0$ inside the repulsive core ($r \lesssim 0.93$) and then oscillates as in a dense liquid. The first peak is at $r = 1.089$ ($g = 3.00$), the first minimum at $r = 1.544$ ($g = 0.576$), and further peaks at $r = 2.079$ ($g = 1.28$) and $r = 2.985$ ($g = 1.11$). The oscillations die out and $g(L/2) = 0.993 \approx 1$: there is short-range order but no long-range order.

- **Energy:** $\langle E_{\mathrm{tot}} \rangle / N = -4.98$ per particle, and the tail is 9% of E_{tot}.

- **Pressure:** $\langle P_{\mathrm{kin}} \rangle = 0.5945 \approx \rho^* \langle T^* \rangle = 0.596$. The repulsive virial 0.100 and the attractive tail -0.746 almost cancel it, so $P_{\mathrm{tot}} = -0.052$ is close to zero. Without the tail P^* would be $+0.69$, so the tail correction is essential at $r_c = 2.5$.

- **Convergence:** $\langle E_{\mathrm{tot}} \rangle$ changes by only 0.2 (less than 0.01%) once a few thousand steps have been averaged. $\langle P_{\mathrm{tot}} \rangle$ converges more slowly: it is a small difference of large terms. It settles after about 5000 steps ($t^* \approx 23$) to within ± 0.006.

3 Exercise 3

3.1 Question

STEP 3: Analysing the dynamics.

- Modify the code to calculate the velocity autocorrelation function (VACF) and the mean square displacement (MSD):

$$C(t) = \langle \vec{v}(t) \cdot \vec{v}(0) \rangle, \tag{3}$$

$$r^2(t) = \langle [\vec{r}(t) - \vec{r}(0)]^2 \rangle. \tag{4}$$

  Compute the VACF and the MSD for a run of 10000 steps. Plot the results.

- Write a code to calculate the spectral density (SDW) of the VACF:

$$S(\omega) = \int_0^\infty dt\, C(t) \cos(\omega t). \tag{5}$$

  Use a Simpson 3/8 rule or similar. Plot the results.

- Use Einstein's diffusion formula to obtain the self-diffusion coefficient D:

$$D = \lim_{t\to\infty} \frac{1}{6t} r^2(t). \tag{6}$$

3.2 Code

Only the lines added in this step are shown, with a few lines of the existing program kept where they mark the insertion point. The full program is listed in the Annex.

```
11      ! 1. Defining dimensions
12      dimension r(3,1000), vinf(3,1000), accel(3,1000), runw(3,1000)
13      parameter (nhis = 1000)
14      dimension g(nhis)
15      ! VACF and MSD accumulated over multiple time origins (circular buffer)
16      integer, allocatable :: time0(:), ntime(:)
17      double precision, allocatable :: x0(:,:,:), v0(:,:,:), vacf(:), r2t(:)
```

Listing 9: Unwrapped positions `runw` and storage for the time origins, the VACF and the MSD.

```
51      ! runw holds positions without periodic wrapping, needed for the MSD.
52      runw = r
```

Listing 10: Unwrapped coordinates, initialised once the positions are in reduced units.

```
70      sumt2 = 0.d0
71      sume = 0.d0
72      sume2 = 0.d0
73
74      ! Window of ~5 time units (VACF decayed, MSD linear), ncor a multiple of 6.
75      ! A new time origin every it0 steps; n0max origins fit in the window.
76      it0 = 5
77      ncor = 6*(min(nint(5.d0/deltat), nconf)/6)
78      n0max = ncor/it0 + 1
79      allocate(time0(n0max), x0(3,natoms,n0max), v0(3,natoms,n0max))
80      allocate(ntime(0:ncor), vacf(0:ncor), r2t(0:ncor))
```

Listing 11: Correlation window and circular buffer of time origins.

```
   do i = 1, nconf
      if (mod(i, 1000) == 0) then
         print *, '  -> Completado paso ', i, ' de ', nconf
      end if
      call forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
      call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, boxlength)

      ! Leap-frog displacement is vinf*deltat; added without PBC.
      runw(:,1:natoms) = runw(:,1:natoms) + vinf(:,1:natoms)*deltat

      ! Store a new time origin, overwriting the oldest one.
      if (mod(i, it0) == 0) then
         n0 = n0 + 1
         k = mod(n0-1, n0max) + 1
         time0(k) = i
         x0(:,:,k) = runw(:,1:natoms)
         v0(:,:,k) = vinf(:,1:natoms)
      end if
      ! Accumulate v(t).v(0) and [r(t)-r(0)]^2 for every stored origin.
      do k = 1, min(n0, n0max)
         it = i - time0(k)
         if (it <= ncor) then
            ntime(it) = ntime(it) + 1
```

Listing 12: After each leap-frog step the unwrapped positions are advanced, a time origin is stored every 5 steps, and $C(t)$ and $r^2(t)$ are accumulated over all stored origins.

```
   print *, '  Etot  =', etot_avg
   print *, '  Pkin  =', pkin_avg
   print *, '  Pvir  =', pvir_avg
   print *, '  Ptail =', ptail
   print *, '  Ptot  =', ptot_avg

   ! 6. Radial distribution function in reduced units.
   open(15, file='g-leap.dat', status='unknown')
   do j = 1, nhis
      rr = delg*(dfloat(j) - 0.5d0)
      vb = (dfloat(j)**3 - dfloat(j-1)**3)*delg**3
      nid = (4.d0/3.d0)*pi*vb*rho
      g(j) = g(j)/(dfloat(nconf)*dfloat(natoms)*nid)
      write(15,*) rr, g(j)
   end do
   close(15)

   ! 7. VACF and MSD averaged over atoms and time origins.
   vacf = vacf/(dfloat(natoms)*ntime)
   r2t = r2t/(dfloat(natoms)*ntime)
   open(16, file='c-leap.dat', status='unknown')
   open(17, file='r2-leap.dat', status='unknown')
   open(19, file='D-leap.dat', status='unknown')
   do it = 0, ncor
      write(16,*) it*deltat, vacf(it)
      write(17,*) it*deltat, r2t(it)
      if (it > 0) write(19,*) it*deltat, r2t(it)/(6.d0*it*deltat)
   end do
   close(16)
   close(17)
   close(19)
   print *, '  C(0) =', vacf(0)

   ! 8. Spectral density S(w) = integral C(t) cos(w t) dt, w = 0, ..., 100.
   !    Composite Simpson 3/8 rule on the first quarter of the window
   !    (C(t) ~ 0 beyond), with a multiple of 3 intervals.
```

Listing 13: Normalisation and output of the VACF and the MSD, and spectral density of $C(t)$ by the composite Simpson 3/8 rule.

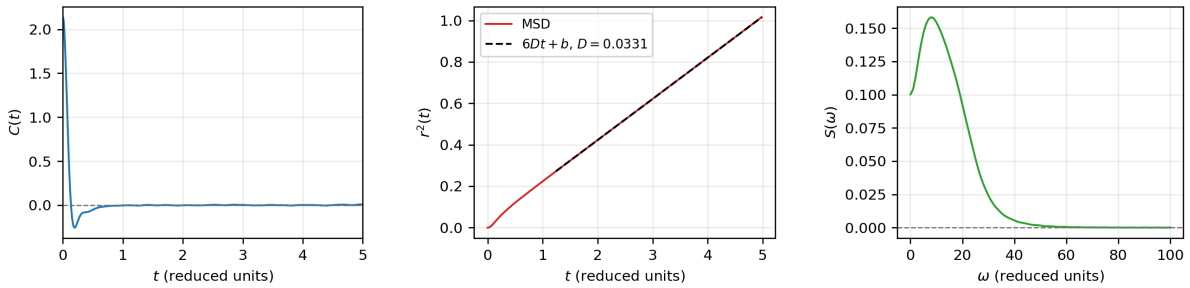
```
    open(18, file='c_spectrum-leap.dat', status='unknown')
    do iw = 0, 100
       s = 0.d0
       do it = 0, nsdw
          if (it == 0 .or. it == nsdw) then
             wgt = 1.d0
          else if (mod(it, 3) == 0) then
             wgt = 2.d0
          else
             wgt = 3.d0
          end if
          s = s + wgt*vacf(it)*dcos(iw*it*deltat)
       end do
       write(18,*) dfloat(iw), 3.d0*deltat/8.d0*s
    end do
    close(18)

    ! 9. Einstein: least-squares fit r2(t) = 6*D*t + b over the diffusive
    !    regime (the first quarter of the window, ballistic, is discarded).
    !    yint is used instead of "intercept": implicit typing would make it integer.
    ifit = ncor/4
    npts = ncor - ifit + 1
    sx = 0.d0
    sy = 0.d0
    sxx = 0.d0
```

Listing 14: Einstein self-diffusion coefficient from a linear fit of $r^2(t)$ in the diffusive regime.

3.3 Results

On the same 10000-step run as in Exercise 2 ($N = 500$, $\rho^* = 0.835$, $\langle T^* \rangle = 0.713$), $C(t)$ and $r^2(t)$ were averaged over all atoms and over many starting times, in a window of 1074 steps ($t^* \leq 4.98$). Figure 3 shows the VACF, the MSD with its straight-line (Einstein) fit, and the spectral density $S(\omega)$. The fit gives a slope of 0.1988 and an intercept of 0.025, so $D = 0.0331$.



(a) VACF (b) MSD (c) SDW

Figure 3: Dynamics of the Lennard-Jones liquid in reduced units. (a) Velocity autocorrelation $C(t)$. (b) Mean square displacement $r^2(t)$; the dashed line is the fit $r^2 = 6Dt + b$ on $t^* \in [1.24, 4.98]$. (c) Spectral density $S(\omega)$ of the VACF, Simpson 3/8 rule, $\omega = 0, 1, \ldots, 100$.

3.4 Discussion & Notes

- **Results vs. expectations:**
  - *VACF:* The initial value $C(0) = 2.138$ shows excellent agreement with the equipartition theorem, which yields a mean-square velocity of $\langle v^2 \rangle = 3\langle T^* \rangle = 2.139$ (for $m = 1$). The velocity autocorrelation function $C(t)$ decays rapidly thereafter, retaining only 21% of its initial value by $t^* = 0.1$. It crosses zero at $t^* = 0.139$ (0.30 ps) and reaches a minimum of $C = -0.256$ at $t^* = 0.195$ (0.42 ps). This negative correlation indicates backscattering: on average, a particle reverses its velocity after colliding with the transient "cage" formed by its nearest neighbors (corresponding to the first radial distribution peak at $r = 1.09$ in Exercise 2). Such rebound behavior is characteristic of dense liquids, in contrast to dilute gases where $C(t)$ decays monotonically toward zero. By $t^* \approx 0.64$, $|C| < 0.02$, signifying that the atom has effectively lost memory of its initial velocity.
  - *MSD:* At very short timescales ($t^* \lesssim 0.05$), atomic motion is purely ballistic, satisfying $\langle \Delta r^2(t) \rangle \approx \langle v^2 \rangle t^2$. Beyond this regime, frequent collisions deflect the particle, initiating a crossover into the diffusive regime; for $t^* \gtrsim 1$, $\langle \Delta r^2(t) \rangle$ scales linearly with time, consistent with Brownian motion.
  - *Direct Einstein ratio:* Calculating the instantaneous ratio $\langle \Delta r^2(t) \rangle / (6t)$ from (6) yields 0.0372 at $t^* = 1$ and 0.0340 at $t^* = 4.98$. Although this quotient asymptotically approaches the true diffusion coefficient, it systematically overestimates D at finite times because it does not decouple the early-time ballistic displacement. A linear regression over the diffusive regime ($t^* \gtrsim 1$) resolves this issue: the fitted slope accurately extracts $D = 0.0331$, while the non-zero intercept absorbs the ballistic offset.
  - *Spectral density $S(\omega)$:* The velocity power spectrum characterizes the distribution of vibrational modes, exhibiting a prominent peak at $\omega = 8$ ($S = 0.158$) and decaying sharply at high frequencies ($S = 5.4 \times 10^{-3}$ at $\omega = 40$; 5×10^{-5} at $\omega = 100$). Crucially, the non-zero value at the origin, $S(0) = 0.100$, indicates macroscopic self-diffusion. As a result, we can relate the self-diffusion coeficient with the spectral density at 0 using the Green–Kubo formula:

    $$D = \frac{1}{3} \int_0^\infty C(t)\, dt = \frac{S(0)}{3} = 0.0334 \approx 0.0331, \tag{7}$$

    demonstrating strong consistency with the MSD linear fit.

- **Conclusion:** The backscattering minimum in $C(t)$, the soft vibrational peak of $S(\omega)$ at $\omega = 8$, and the diffusion coefficient $D \approx 0.033$ consistently describe the structural and dynamical properties of a dense Lennard-Jones liquid, corroborating the radial distribution function obtained in Exercise 2.

Annex

The program listed here is the full source used throughout the practice. It is the leap-frog integrator supplied for Exercise 1, together with the radial distribution function, the virial pressure and the tail corrections added in Exercise 2, and the velocity autocorrelation, mean square displacement, spectral density and self-diffusion coefficient added in Exercise 3. Each exercise section quotes only the lines introduced in that step.

```
!************************************************************************
!          Molecular Dynamics code to simulate at NVE ensemble
!          a system of N atoms of Ar inside a cubic box, interacting
!          through a truncated Lennard-Jones force field at 2.5 sigma.
!          Leap-frog Verlet integration algorithm.
!************************************************************************
program leapfroglj
   implicit double precision(a-h,o-z)
   double precision mass, nid

   ! 1. Defining dimensions
   dimension r(3,1000), vinf(3,1000), accel(3,1000), runw(3,1000)
   parameter (nhis = 1000)
   dimension g(nhis)
   ! VACF and MSD accumulated over multiple time origins (circular buffer)
   integer, allocatable :: time0(:), ntime(:)
   double precision, allocatable :: x0(:,:,:), v0(:,:,:), vacf(:), r2t(:)

   INCLUDE '../../chdir_to_code.inc'

   ! 2. Reading data and computing related quantities
   open(1, file='../data/leap-lj.data', status='old')
   read(1,*) nconf
   read(1,*) natoms
   read(1,*) mass
   read(1,*) sigma, epsil
   read(1,*) deltat
   close(1)

   nf = 3*natoms - 3  ! number of degrees of freedom
   rc = 2.5d0         ! cutoff radius in units of sigma

   ! 3. Reading initial configuration (positions, velocities) in A and A/ps
   open(2, file='../data/leap-conf.data', status='old')
   do is = 1, natoms
      read(2,*) (r(l,is), l=1,3)
      read(2,*) (vinf(l,is), l=1,3)
   end do
   read(2,*) boxlength
   close(2)

   ! Opening files to write results
   open(3, file='energy-leap.dat', status='unknown')
   open(4, file='temp-leap.dat', status='unknown')
   open(8, file='thermo-leap.dat', status='unknown')

   ! 4. Change to reduced units
   call reduced(natoms, r, vinf, boxlength, deltat, epsil, sigma, &
                mass, uvel)

   ! runw holds positions without periodic wrapping, needed for the MSD.
   runw = r

   ! 5. Start the loop to generate new configurations
   pi = 4.d0*datan(1.d0)
   delg = boxlength/(2.d0*dfloat(nhis))
   rho = dfloat(natoms)/boxlength**3
```

```
    etail = (8.d0*pi/3.d0)*rho*dfloat(natoms)* &
            (1.d0/(3.d0*rc**9) - 1.d0/rc**3)
    ptail = (16.d0*pi/3.d0)*rho**2* &
            (2.d0/(3.d0*rc**9) - 1.d0/rc**3)
    do j = 1, nhis
       g(j) = 0.d0
    end do
    ekin_sum = 0.d0
    epot_sum = 0.d0
    pkin_sum = 0.d0
    pvir_sum = 0.d0
    sumt = 0.d0
    sumt2 = 0.d0
    sume = 0.d0
    sume2 = 0.d0

    ! Window of ~5 time units (VACF decayed, MSD linear), ncor a multiple of 6.
    ! A new time origin every it0 steps; n0max origins fit in the window.
    it0 = 5
    ncor = 6*(min(nint(5.d0/deltat), nconf)/6)
    n0max = ncor/it0 + 1
    allocate(time0(n0max), x0(3,natoms,n0max), v0(3,natoms,n0max))
    allocate(ntime(0:ncor), vacf(0:ncor), r2t(0:ncor))
    ntime = 0
    vacf = 0.d0
    r2t = 0.d0
    n0 = 0

    write(8,'(a)') '# t etot pkin pvir ptail ptot'

    print *, 'Iniciando simulacion MD (', nconf, ' pasos)...'
    do i = 1, nconf
       if (mod(i, 1000) == 0) then
          print *, '  -> Completado paso ', i, ' de ', nconf
       end if
       call forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
       call velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, boxlength)

       ! Leap-frog displacement is vinf*deltat; added without PBC.
       runw(:,1:natoms) = runw(:,1:natoms) + vinf(:,1:natoms)*deltat

       ! Store a new time origin, overwriting the oldest one.
       if (mod(i, it0) == 0) then
          n0 = n0 + 1
          k = mod(n0-1, n0max) + 1
          time0(k) = i
          x0(:,:,k) = runw(:,1:natoms)
          v0(:,:,k) = vinf(:,1:natoms)
       end if
       ! Accumulate v(t).v(0) and [r(t)-r(0)]^2 for every stored origin.
       do k = 1, min(n0, n0max)
          it = i - time0(k)
          if (it <= ncor) then
             ntime(it) = ntime(it) + 1
             vacf(it) = vacf(it) + sum(vinf(:,1:natoms)*v0(:,:,k))
             r2t(it) = r2t(it) + sum((runw(:,1:natoms) - x0(:,:,k))**2)
          end if
       end do

       ! Conserved NVE energy (no tail). Thermodynamic totals include it.
       etot = ecin + epot
       etot_corr = etot + etail
       pkin = 2.d0*ecin/(3.d0*boxlength**3)
```

```
121         pvir = vir/(3.d0*boxlength**3)
122         ptot = pkin + pvir + ptail
123         ekin_sum = ekin_sum + ecin
124         epot_sum = epot_sum + epot
125         pkin_sum = pkin_sum + pkin
126         pvir_sum = pvir_sum + pvir
127         write(3,*) i*deltat, etot
128         write(4,*) i*deltat, temp
129         write(8,*) i*deltat, etot_corr, pkin, pvir, ptail, ptot
130         sumt = sumt + temp
131         sumt2 = sumt2 + temp*temp
132         sume = sume + etot
133         sume2 = sume2 + etot*etot
134      end do
135      close(3)
136      close(4)
137      close(8)
138
139      ! Mean and standard deviation of T and E_tot over the run
140      tmean = sumt/nconf
141      emean = sume/nconf
142      tsd = dsqrt(sumt2/nconf - tmean**2)
143      esd = dsqrt(sume2/nconf - emean**2)
144      print *, 'steps  = ', nconf
145      print *, '<T>    = ', tmean, '  sigma_T = ', tsd
146      print *, '<Etot> = ', emean, '  sigma_E = ', esd
147
148      ekin_avg = ekin_sum/dfloat(nconf)
149      epot_avg = epot_sum/dfloat(nconf)
150      etot_avg = ekin_avg + epot_avg + etail
151      pkin_avg = pkin_sum/dfloat(nconf)
152      pvir_avg = pvir_sum/dfloat(nconf)
153      ptot_avg = pkin_avg + pvir_avg + ptail
154
155      open(9, file='averages-leap.dat', status='unknown')
156      write(9,*) 'nconf', nconf
157      write(9,*) 'ekin', ekin_avg
158      write(9,*) 'epot', epot_avg
159      write(9,*) 'etail', etail
160      write(9,*) 'etot', etot_avg
161      write(9,*) 'pkin', pkin_avg
162      write(9,*) 'pvir', pvir_avg
163      write(9,*) 'ptail', ptail
164      write(9,*) 'ptot', ptot_avg
165      close(9)
166      print *, 'Mean values in reduced units'
167      print *, '  Ekin  =', ekin_avg
168      print *, '  Epot  =', epot_avg
169      print *, '  Etail =', etail
170      print *, '  Etot  =', etot_avg
171      print *, '  Pkin  =', pkin_avg
172      print *, '  Pvir  =', pvir_avg
173      print *, '  Ptail =', ptail
174      print *, '  Ptot  =', ptot_avg
175
176      ! 6. Radial distribution function in reduced units.
177      open(15, file='g-leap.dat', status='unknown')
178      do j = 1, nhis
179         rr = delg*(dfloat(j) - 0.5d0)
180         vb = (dfloat(j)**3 - dfloat(j-1)**3)*delg**3
181         nid = (4.d0/3.d0)*pi*vb*rho
182         g(j) = g(j)/(dfloat(nconf)*dfloat(natoms)*nid)
183         write(15,*) rr, g(j)
```

```
   end do
   close(15)

   ! 7. VACF and MSD averaged over atoms and time origins.
   vacf = vacf/(dfloat(natoms)*ntime)
   r2t = r2t/(dfloat(natoms)*ntime)
   open(16, file='c-leap.dat', status='unknown')
   open(17, file='r2-leap.dat', status='unknown')
   open(19, file='D-leap.dat', status='unknown')
   do it = 0, ncor
      write(16,*) it*deltat, vacf(it)
      write(17,*) it*deltat, r2t(it)
      if (it > 0) write(19,*) it*deltat, r2t(it)/(6.d0*it*deltat)
   end do
   close(16)
   close(17)
   close(19)
   print *, '  C(0) =', vacf(0)

   ! 8. Spectral density S(w) = integral C(t) cos(w t) dt, w = 0, ..., 100.
   !    Composite Simpson 3/8 rule on the first quarter of the window
   !    (C(t) ~ 0 beyond), with a multiple of 3 intervals.
   nsdw = 3*(ncor/12)
   open(18, file='c_spectrum-leap.dat', status='unknown')
   do iw = 0, 100
      s = 0.d0
      do it = 0, nsdw
         if (it == 0 .or. it == nsdw) then
            wgt = 1.d0
         else if (mod(it, 3) == 0) then
            wgt = 2.d0
         else
            wgt = 3.d0
         end if
         s = s + wgt*vacf(it)*dcos(iw*it*deltat)
      end do
      write(18,*) dfloat(iw), 3.d0*deltat/8.d0*s
   end do
   close(18)

   ! 9. Einstein: least-squares fit r2(t) = 6*D*t + b over the diffusive
   !    regime (the first quarter of the window, ballistic, is discarded).
   !    yint is used instead of "intercept": implicit typing would make it integer.
   ifit = ncor/4
   npts = ncor - ifit + 1
   sx = 0.d0
   sy = 0.d0
   sxx = 0.d0
   sxy = 0.d0
   do it = ifit, ncor
      tt = it*deltat
      sx = sx + tt
      sy = sy + r2t(it)
      sxx = sxx + tt*tt
      sxy = sxy + tt*r2t(it)
   end do
   slope = (dfloat(npts)*sxy - sx*sy)/(dfloat(npts)*sxx - sx*sx)
   yint = (sy - slope*sx)/dfloat(npts)
   d_einstein = slope/6.d0

   open(20, file='diffusion-leap.dat', status='unknown')
   write(20,'(a)') '# D_einstein slope intercept tmin tmax'
   write(20,*) d_einstein, slope, yint, ifit*deltat, ncor*deltat
```

```
247     close(20)
248     print *, '  D (Einstein fit, reduced units) =', d_einstein
249
250     ! 10. Saving last configuration in A and A/ps
251     open(11, file='newconf.data', status='unknown')
252     do is = 1, natoms
253        write(11,*) (r(l,is)*sigma, l=1,3)
254        write(11,*) (vinf(l,is)*uvel, l=1,3)
255     end do
256     write(11,*) boxlength*sigma
257     close(11)
258
259     stop
260   end program leapfroglj
261
262   !*************************************************************
263   !              subroutine reduced
264   !*************************************************************
265
266   subroutine reduced(natoms, r, vinf, boxlength, deltat, &
267                      epsil, sigma, mass, uvel)
268     implicit double precision(a-h,o-z)
269     double precision mass
270     dimension r(3,1000), vinf(3,1000)
271
272     rgas = 8.314472673d0  ! J/(mol*K)
273     utime = sigma*dsqrt(mass/epsil)*dsqrt(10.d0/rgas)
274     uvel = sigma/utime    ! unit of velocity, expressed in A/ps
275
276     boxlength = boxlength/sigma
277     deltat = deltat/utime
278     do is = 1, natoms
279        do l = 1, 3
280           r(l,is) = r(l,is)/sigma
281           vinf(l,is) = vinf(l,is)/uvel
282        end do
283     end do
284
285     return
286   end subroutine reduced
287
288   !*************************************************************
289   !              subroutine forces
290   !*************************************************************
291
292   subroutine forces(natoms, r, boxlength, accel, rc, epot, vir, nhis, g, delg)
293     implicit double precision(a-h,o-z)
294     dimension r(3,1000), accel(3,1000)
295     dimension g(nhis)
296
297     do is = 1, natoms
298        do l = 1, 3
299           accel(l,is) = 0.d0  ! sets accelerations to 0
300        end do
301     end do
302     epot = 0.d0
303     vir = 0.d0
304
305     ! atom-atom interactions
306     do is = 1, natoms-1
307        do js = is+1, natoms
308           call lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
309           epot = epot + pot
```

```
310           vir = vir + virij
311        end do
312     end do
313
314     return
315   end subroutine forces
316
317   !*************************************************************
318   !              subroutine Lennard-Jones
319   !*************************************************************
320
321   subroutine lj(is, js, r, boxlength, accel, rc, pot, virij, nhis, g, delg)
322     implicit double precision(a-h,o-z)
323     dimension r(3,1000), accel(3,1000)
324     dimension rij(3)
325     dimension g(nhis)
326
327     rr2 = 0.d0
328     pot = 0.d0
329     virij = 0.d0
330
331     do l = 1, 3
332        rijl = r(l,js) - r(l,is)
333        rij(l) = rijl - boxlength*dnint(rijl/boxlength)
334        rr2 = rr2 + rij(l)*rij(l)
335     end do
336
337     rr = dsqrt(rr2)
338
339     if (rr .lt. rc) then
340        ynvrr2 = 1.d0/rr2
341        ynvrr6 = ynvrr2*ynvrr2*ynvrr2
342        ynvrr12 = ynvrr6*ynvrr6
343        forcedist = 24.d0*(2.d0*ynvrr12 - ynvrr6)*ynvrr2
344        pot = 4.d0*(ynvrr12 - ynvrr6)
345        do l = 1, 3
346           accel(l,is) = accel(l,is) - forcedist*rij(l)
347           accel(l,js) = accel(l,js) + forcedist*rij(l)
348        end do
349        virij = forcedist*rr2
350     end if
351
352     ! Histogram of pair distances up to L/2 (minimum image).
353     if (rr .lt. boxlength/2.d0) then
354        ig = int(rr/delg) + 1
355        if (ig .ge. 1 .and. ig .le. nhis) g(ig) = g(ig) + 2.d0
356     end if
357
358     return
359   end subroutine lj
360
361   !*************************************************************
362   !              subroutine velpos
363   !*************************************************************
364
365   subroutine velpos(natoms, vinf, accel, deltat, r, nf, ecin, temp, &
366                     boxlength)
367     implicit double precision(a-h,o-z)
368     dimension vinf(3,1000), accel(3,1000), r(3,1000)
369
370     ecin = 0.d0
371     do is = 1, natoms
372        v2 = 0.d0
```

```
373         do l = 1, 3
374            vsup = vinf(l,is) + accel(l,is)*deltat
375            r(l,is) = r(l,is) + vsup*deltat
376            v = (vsup + vinf(l,is))/2.d0
377            v2 = v2 + v*v
378            vinf(l,is) = vsup
379         end do
380         ecin = ecin + 0.5d0*v2
381      end do
382      temp = 2.d0*ecin/dfloat(nf)
383
384      ! Applying periodic boundary conditions
385      do is = 1, natoms
386         do l = 1, 3
387            if (r(l,is) .lt. 0.d0) r(l,is) = r(l,is) + boxlength
388            if (r(l,is) .gt. boxlength) r(l,is) = r(l,is) - boxlength
389         end do
390      end do
391
392      return
393   end subroutine velpos
```

Listing 15: Full leap-frog program used for the three exercises.